STREAMING INTEGRATION AGENT BRIEF

Complete Technical Specification for a Scraper + Player
that replaces ScreenScape inside Allrated

TARGET PLATFORM

Allrated - React 18 + Vite + TypeScript client
Node/Express + Prisma + PostgreSQL server on Railway

WHAT THIS BRIEF COVERS

Site scraping architecture * Stream extraction
Custom HLS video player * Full backend + frontend
integration steps * Error handling + fallback chains

AGENT REQUIREMENTS

Node.js / Python 3 * Playwright * ffprobe
Reverse-engineering browser DevTools knowledge
Basic HLS / MPEG-DASH understanding

Table of Contents

§1 Mission Objective & Architecture Overview

What you are building and why

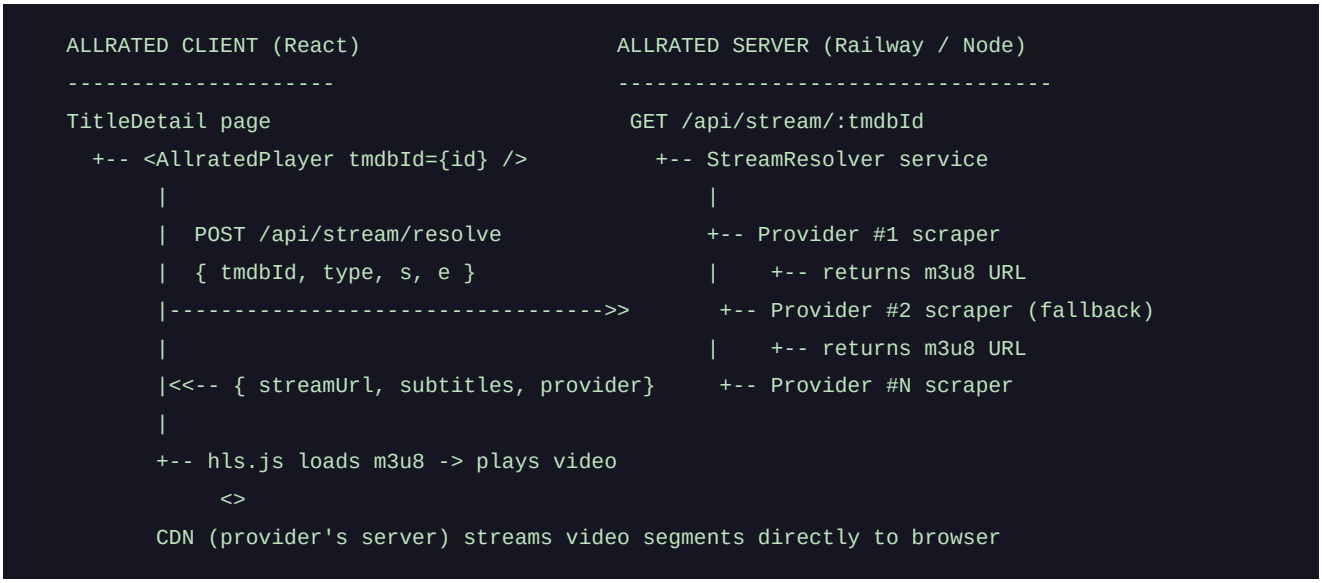
You are building a self-hosted streaming layer for the Allrated platform. Currently, Allrated embeds the third-party ScreenScape player inside an <iframe>. This gives Allrated no control over the player UI, no control over ads, no analytics, no brand consistency, and no fallback when ScreenScape goes down. Your job is to replace that iframe entirely with Allrated's own scraper backend and a custom video player - giving full ownership of the streaming experience.

What You Will Build

- A Node.js (or Python) scraper service that accepts a TMDB ID + type + season + episode and returns one or more direct stream URLs (HLS .m3u8 or MP4).
- A fallback chain that tries Provider A -> Provider B -> ... -> Provider N until a working stream is found.
- A custom React HLS video player built on hls.js with full controls - seeking, quality selection, subtitle tracks, playback speed, picture-in-picture, and resume.
- Backend Express routes at /api/stream/:tmdbId that proxy or return stream metadata.
- Frontend integration: replacing the existing <ScreenScapePlayer /> component inside client/src/components/player/ with the new <AllratedPlayer /> component.

High-Level Architecture Diagram

System Architecture



CRITICAL

The scraper runs SERVER-SIDE on Railway. The client never talks to piracy sites directly. This keeps piracy-site traffic invisible to the browser, avoids CORS errors, and lets you add caching + rate-limiting on your own infrastructure.

Technology Stack for This Build

Layer	Technology	Purpose

Scraper Core	Node.js + Axios + Cheerio	HTTP fetching + HTML parsing for simple sites
JS-rendered sites	Playwright (Python or Node)	Full browser automation for sites that need JS
Reverse engineering	Python 3 + mitmproxy / httpx	Decode obfuscated URLs, JS crypto, HAR analysis
Stream extraction	yt-dlp (Python CLI)	Universal extractor - handles 1000+ hosters
Video player	React + hls.js + Plyr or custom	Client-side HLS playback with full controls
Backend routes	Express + node-cache	API layer with 6h stream URL caching
Anti-bot evasion	Playwright stealth + FingerprintJS	Bypass Cloudflare, PerimeterX, BotD
Subtitles	OpenSubtitles API + VTT parser	Fetch + inject .vtt subtitle tracks

§2 Provider Roster - All 35+ Streaming Sources

Complete list extracted from ScreenScape's providers.json (github.com/Anshu78780/json)

These are every provider that ScreenScape queries. Your scraper will work through this list in priority order. Providers are grouped by content specialty. URLs rotate frequently - always read the canonical URL from a config file, never hardcode.

Group A - Indian / South Asian Content (Highest Priority for Hindi/Tamil/Telugu)

Key	Site Name	Current Domain	Content Type	Difficulty
hdhub	HDHub4U	new3.hdhub4u.cl	Bollywood + Hollywood	Medium
4kHDHub	4kHDHub	4khdhub.one	(Hindi dub)	Medium
Moviesmod	Moviesmod	moviesmod.farm	Hindi + English	Easy
Topmovies	Topmovies	moviesleech.bar	Hindi + multi-audio	Easy
UhdMovies	UhdMovies	uhdmovies.food	UHD/4K content	Medium
drive	MoviesDrive	new6.moviesdrives.my	Hindi dub drives	Medium
multi	MultiMovies	multimovies.makeup	Multi-audio packs	Easy
w4u	World4uFree	world4ufree.im	Hindi + English	Easy
filmyfly	FilmyFly	new2.filmyfiy.org	Bollywood + OTT rips	Easy
KMMovies	KMMovies	kmmovies.mom	Hindi + South dub	Medium
	DesiReMovies	1desiremovies.mov	Desi content	Easy
	AllMoviesHub	allmovieshub.doctor	Hindi + multi-lang	Easy
skymovieshd	SkyMoviesHD	skymovieshd.ceo	HD Hindi + OTT	Easy
yomovies	YoMovies	yomovies.ventures	Hindi + South dub	Easy
movies4u	Movies4U	movies4u.ar	Hindi + English	Easy
movies4u2	Movies4U (alt)	movies4u.ee	Hindi + English	Easy
zinkmovies	ZinkMovies	zinkmovies.cc	Hindi + English	Medium
filmyclub	FilmyClub	filmycab.co	Hindi OTT rips	Easy
oggmovies	OggMovies	ogomovies.dev	Hindi + regional	Easy
zeefliz	ZeeFliz	zeefliz.beer	Indian OTT mirror	Hard
Vega	VegaMovies	vegamovies.navy	4K + Hindi + English	Medium
cinemaLuxe	CinemaLuxe	new1.cinemalux.foo	Hindi + English	Easy
themoviesflix	ThemoviesFlix	themoviesflix.llc	Hindi + English	Easy
srmovies	SRMovies (ssrmovies)	ssrmovies.luxe	Hindi + English	Easy

Group B - Global / International Sources

Key	Site Name	Current Domain	Content Type	Difficulty

rive	Rive (RiveStream)	watch.rivestream.app	Global - multi-provider aggregator	Easy
nfMirror	NFMirror	net22.cc	Netflix mirror / OTT rips	Hard
watchulz	WatchRulz	watchmovierulz.cafe	Telugu + Tamil + Hindi	Medium
joya9tv	Joya9TV	joya9tv1.com	Live TV + Bangla content	Hard

Group C - Anime Sources

Key	Site Name	Current Domain	Content Type	Difficulty
consumet	Consumet API	consumet.zendax.tech	Anime - REST API (no scraping needed)	Easy
kissanime	KissAnime	kissanime.co	Sub + dub embedded anime	Hard
animesalt	AnimeSalt	animesalt.link	Anime streaming	Medium
	TheAnimeWorld	watchanimeworld.net	Anime streaming	Medium
tokyoinsider	TokyoInsider	tokyoinsider.com	Classic + current anime	Easy

Group D - Korean Drama / Asian Content

Key	Site Name	Current Domain	Content Type	Difficulty
kissKh	KissKH	kisskh.ws	K-drama + C-drama + Thai drama	Medium
dramafull	DramaFull	dramafull.cc	Asian dramas with subtitles	Medium

WARNING

Domain names change frequently (every few weeks). Build a config-driven provider registry (JSON file or database table) so you can update domains without redeploying code. ScreenScape stores theirs at: github.com/Anshu78780/json/blob/main/providers.json

§3 Scraper Architecture - How to Extract Streams

The four-stage pipeline every provider goes through

Every streaming site, regardless of how it looks on the outside, follows the same underlying data flow. There are four stages: (1) resolve the title page URL from a TMDB ID, (2) extract the embed/player page URL from the title page, (3) extract the raw video host URL from the player page, and (4) resolve the actual m3u8 or MP4 from the video host. You must implement each stage - skipping any stage breaks the chain.

Stage 1 - TMDB ID -> Title Search URL

Most piracy sites do NOT store TMDB IDs. You must convert the TMDB ID to a title + year, then search the provider's own search endpoint.

HTTP Flow

```
# Step 1a: Resolve TMDB ID to title + year
GET https://api.themoviedb.org/3/movie/{tmdbId}?api_key={TMDB_KEY}
  -> { title: 'Inception', release_date: '2010-07-16', imdb_id: 'tt1375666' }

# Step 1b: Search the provider site
GET https://hdhub4u.cl/?s=Inception+2010
  -> HTML page with search results
  -> Parse: find <a href> containing 'inception' in the URL

# Step 1c: Alternative - use IMDB ID directly (some sites accept it)
GET https://example.com/imdb/tt1375666
```

Stage 2 - Title Page -> Player Embed URL

Once on the title page, locate the embedded video player. This is almost always an <iframe> whose src points to a video CDN host. The iframe src is what you need from this stage.

Python - Stage 2

```
# Parse HTML for iframe embeds
soup = BeautifulSoup(html, 'html.parser')
iframes = soup.find_all('iframe')
# Look for known CDN hostnames in src attributes:
# StreamWish, Filelions, Streamtape, DoodStream, StreamSB,
# VidHide, VidPlay, MixDrop, UpStream, VidGuard, StreamVid
for iframe in iframes:
    src = iframe.get('src', '') or iframe.get('data-src', '')
    if any(host in src for host in KNOWN_CDN_HOSTS):
        player_url = src
        break
```

Stage 3 - Player Page -> Video Host URL Extraction

CRITICAL

This is the hardest stage. Most player pages obfuscate the real video URL using:

a) Base64 encoding b) AES-128 encryption with a hardcoded key

c) JavaScript eval() obfuscation d) Dynamic XHR requests on page load

You must run the page in Playwright (headless browser) OR reverse-engineer the JS.

Method A - Playwright Network Interception (Most Reliable)

Python - Playwright Interception

```python
from playwright.async_api import async_playwright
import asyncio

async def extract_stream_url(player_url: str) -> str:
    async with async_playwright() as p:
        browser = await p.chromium.launch(
            headless=True,
            args=['--disable-blink-features=AutomationControlled']
        )
        context = await browser.new_context(
            user_agent='Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...',
            extra_http_headers={'Referer': player_url}
        )
        page = await context.new_page()

        stream_url = None

        # Intercept all network requests
        async def handle_request(request):
            nonlocal stream_url
            url = request.url
            # .m3u8 or direct mp4 = the stream
            if '.m3u8' in url or (url.endswith('.mp4') and 'cdn' in url):
                stream_url = url

        page.on('request', handle_request)
        await page.goto(player_url, wait_until='networkidle', timeout=30000)
        await page.click('video, .play-btn, [class*=play]')
        await asyncio.sleep(5)  # Wait for stream to start

        await browser.close()
        return stream_url
```

Method B - Static JS Reverse Engineering (Faster, Fragile)

Python - JS Extraction

```
import re, base64, jsbeautifier
from Crypto.Cipher import AES

def decode_packed_js(packed_js: str) -> str:
    # Handle p,a,c,k,e,d JavaScript obfuscation
    match = re.search(r"eval\(function\(p,a,c,k,e,d\).*?\}\('(.+?)',(\d+),(\d+),'(.+?)'",
                      packed_js, re.DOTALL)
    if not match:
        return packed_js
    # Use jsbeautifier to unpack
    return jsbeautifier.beautify(packed_js)

def extract_m3u8_from_js(js_code: str) -> str:
    # Common patterns for m3u8 URLs in obfuscated JS
    patterns = [
        r'["\']([^"\']*/master\.m3u8[^"\']*)["\']]',
        r'["\']([^"\']*/index\.m3u8[^"\']*)["\']]',
        r'file:["\'\s]+(https?://[^"\'>]+\.m3u8)',
        r'source:\s*\'(https?://[^\']+\.m3u8)\'',
    ]
    for pattern in patterns:
        match = re.search(pattern, js_code)
        if match:
            return match.group(1)
    return None
```

Stage 4 - yt-dlp Universal Extractor (Fallback for Any Stage)

yt-dlp can handle 1000+ video hosts automatically. Use it as the nuclear option when stages 2-3 are too complex to reverse-engineer.

Python - yt-dlp

```
import subprocess, json

def yt_dlp_extract(url: str, referer: str = None) -> dict:
    cmd = [
        'yt-dlp',
        '--dump-json',
        '--no-download',
        '--no-playlist',
        '--format', 'bestvideo[ext=mp4]+bestaudio/best',
    ]
    if referer:
        cmd += ['--add-header', f'Referer:{referer}']
    cmd.append(url)

    result = subprocess.run(cmd, capture_output=True, text=True, timeout=30)
    if result.returncode == 0:
        data = json.loads(result.stdout)
        return {
            'url': data.get('url'),
            'formats': data.get('formats', []),
            'subtitles': data.get('subtitles', {}),
            'thumbnail': data.get('thumbnail'),
        }
    raise Exception(f'yt-dlp failed: {result.stderr}')
```

§4 Site-by-Site Scraping Playbooks

Exact strategy for each provider group

§4.1 - Indian Piracy Sites (hdhub4u, 4kHDHub, VegaMovies, etc.)

These sites share a common WordPress-based structure with similar HTML patterns. Once you crack one, the others require minor tweaks. They all funnel through a small set of video CDN hosts: StreamWish, Filelions, GDFlix (Google Drive bypass), or Streamtape.

Step-by-step for HDHub4U / VegaMovies / Moviesmod family:

HTTP Flow - Indian Sites

```
1. Search:  GET https://{domain}/?s={title}+{year}
   Parse:   <article class='post'> -> <a href='...'>  (first result)

2. Title page:  GET {article_url}
   Parse:
     Option A: <a href='...'> with text containing '1080p' or 'Download'
               -> this link goes to a GDTOT/GDFlix/DriveHub page
     Option B: <iframe src='https://streamwish.to/e/...'> directly on page

3a. If GDFlix route:  POST https://new.gdflix.dad/api/direct with {id}
    -> returns { download_url: 'https://...' }
    -> That URL goes to a STRM file or direct MP4

3b. If StreamWish route:
    GET https://streamwish.to/e/{id}  (with Referer header set to title page)
    JavaScript on page contains:  jwplayer('vplayer').setup({ sources: [{ file: '...' }] })
    Extract with regex:  r"file\":\"(https?://[^\"]+\.m3u8)\""

4. Return:  { streamUrl, provider: 'hdhub4u', quality: '1080p' }
```

WARNING

Most of these sites serve ads via multiple redirect hops before the real download/stream page.

Use Playwright to follow all redirects automatically. Never follow manually - there can be 4-7 hops.

GDFlix / Google Drive bypass decoder:

Python - GDFlix Decoder

```
import re, requests

def decode_gdflix(gdflix_url: str) -> str:
    # GDFlix uses a short-lived token system
    # Step 1: Load the GDFlix page to get the file ID
    r = requests.get(gdflix_url, headers={'User-Agent': UA})
    file_id = re.search(r'file_id=(\w+)', r.text).group(1)

    # Step 2: POST to their API
    api = 'https://new.gdflix.dad/api/direct'
    resp = requests.post(api, data={'id': file_id},
                         headers={'X-Requested-With': 'XMLHttpRequest'})
    data = resp.json()
    return data.get('download_url') or data.get('url')
```

§4.2 - Anime Sites (consumet, kissanime, tokyoinsider)

Consumet API (No scraping - use their REST API directly):

Consumet API - REST

```
# Consumet is a public REST API  -  no scraping needed
Base: https://consumet.zendax.tech  (or self-host: github.com/consumet/api)

# Search anime
GET /meta/anilist/{title}
  -> { results: [{ id, title, status, episodes, image, ... }] }

# Get episode stream
GET /meta/anilist/watch/{episodeId}
  -> {
       headers: { Referer: '...' },
       sources: [
         { url: 'https://...m3u8', quality: '1080p', isM3U8: true },
         { url: 'https://...m3u8', quality: '720p', isM3U8: true },
       ],
       subtitles: [{ url: '...vtt', lang: 'English' }]
     }

# Map Allrated TMDB anime ID -> Consumet:
GET /meta/tmdb/info/{tmdbId}?type=tv
  -> { episodes: [{ id: 'consumet-ep-id', ... }] }
```

TIP

Consumet is the EASIEST provider. Implement it first and use it as your primary anime source.

You can self-host the Consumet API on Railway to avoid rate limits.

KissAnime (requires Playwright - Cloudflare protected):

Python - KissAnime

```
async def scrape_kissanime(title: str, episode: int) -> str:
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True)
        # Must use stealth plugin to bypass Cloudflare
        await stealth_async(page)  # playwright-stealth

        # Search
        await page.goto(f'https://kissanime.co/anime/search?q={title}')
        result = await page.query_selector('.listing a')
        await result.click()

        # Select episode
        ep_link = await page.query_selector(f'[data-number="{episode}"]')
        await ep_link.click()

        # Intercept m3u8
        stream_url = await intercept_m3u8(page)
        await browser.close()
        return stream_url
```

§4.3 - Korean / Asian Drama (KissKH, DramaFull)

KissKH is the gold standard for K-drama. It has its own API that returns stream URLs directly without complex scraping. DramaFull uses embedded iframes from standard CDN hosts.

HTTP Flow - KissKH

```
# KissKH  -  Has a semi-public API
# Step 1: Get drama list / search
GET https://kisskh.ws/api/DramaList/Search?q={title}&type=2
  -> [{ id, title, episodesCount, ... }]

# Step 2: Get episode list for a drama
GET https://kisskh.ws/api/DramaList/Drama/{dramaId}
  -> { episodes: [{ id, number, title, ... }] }

# Step 3: Get stream URL for an episode
GET https://kisskh.ws/api/DramaList/Episode/{episodeId}.png?err=false&res=0&token=
  -> { Video: 'https://...m3u8', SubTitle: [...] }

# Note: The token param is sometimes validated  -  generate from:
token = base64.b64encode(f'{episodeId}:{timestamp}'.encode()).decode()
```

§4.4 - Global Mirrors (RiveStream, WorldFree4u)

RiveStream is itself a multi-provider aggregator (like ScreenScape). You can treat its embed URL as a stream source - load it in Playwright and intercept the m3u8. WorldFree4u uses standard iframe embeds.

HTTP Flow - Global Mirrors

```
# RiveStream embed  -  just intercept the inner m3u8
RIVE_EMBED = 'https://watch.rivestream.app/embed?tmdb={tmdbId}&type={type}'

# WorldFree4u  -  WordPress-based, same flow as Indian sites
# Search -> title page -> find iframe -> extract from CDN host
```

§5 Reverse Engineering Methodology

How to crack any obfuscated player page

§5.1 - Browser DevTools Workflow

Before writing any code for a new provider, always start with manual DevTools analysis. This tells you exactly what requests the page makes and which one contains the stream URL. Only then write the automated scraper.

- Open the provider site in Chrome. Open DevTools (F12) -> Network tab.
- Filter by: Media (shows video segments), then also XHR (shows API calls), then JS.
- Press play on any video. Watch for requests to .m3u8, .ts, .mp4 files.
- The FIRST .m3u8 request is usually the master playlist - this is your stream URL.
- Right-click -> Copy -> Copy as cURL. Paste into terminal to verify it works.
- Look at the Initiator column - it shows which JS file triggered the request.
- Open that JS file in Sources tab, set a breakpoint on the fetch/XMLHttpRequest call.
- Reload. The breakpoint fires - inspect the call stack to find where the URL is built.
- Search the JS source for the obfuscated string or function that constructs the URL.

TIP

Key insight: even heavily obfuscated pages must make a real HTTP request to load video.
Playwright's request interception catches that request regardless of how complex the JS is.
Use DevTools first to UNDERSTAND the site, then use Playwright to AUTOMATE it.

§5.2 - Python Reverse Engineering Toolkit

Tool 1: mitmproxy - Man-in-the-Middle to capture all traffic

Python - mitmproxy

```
# Install: pip install mitmproxy
# Run as proxy:
mitmdump -w traffic.har --mode regular -p 8080

# Configure system proxy to localhost:8080 (or Playwright via proxy arg)
# Then navigate the streaming site manually
# mitmproxy captures every request including HTTPS

# In Playwright:
browser = await p.chromium.launch(
    proxy={'server': 'http://localhost:8080'}
)

# Parse captured HAR for m3u8 URLs:
import json
har = json.load(open('traffic.har'))
m3u8s = [e['request']['url'] for e in har['log']['entries']
         if '.m3u8' in e['request']['url']]
print(m3u8s)
```

Tool 2: AES Decryption (for encrypted stream tokens)

Python - AES Decryption

```
from Crypto.Cipher import AES
import base64, hashlib

def decrypt_aes_stream_key(encrypted_b64: str, key: str, iv: str) -> str:
    # Many players use AES-CBC with MD5-hashed key
    key_bytes  = hashlib.md5(key.encode()).digest()   # 16 bytes
    iv_bytes   = bytes.fromhex(iv) if len(iv) == 32 else hashlib.md5(iv.encode()).digest()
    cipher     = AES.new(key_bytes, AES.MODE_CBC, iv_bytes)
    decrypted  = cipher.decrypt(base64.b64decode(encrypted_b64))
    # Remove PKCS7 padding
    pad = decrypted[-1]
    return decrypted[:-pad].decode('utf-8')

# Common in StreamSB, StreamVid, VidHide players:
# JS on page does: atob(encrypted_url) -> AES.decrypt(key, iv) -> real m3u8
```

Tool 3: JS Beautifier + Unpacker

Python - JS Unpacker

```
pip install jsbeautifier cchardet

import jsbeautifier, re

def unpack_obfuscated_js(source: str) -> str:
    opts = jsbeautifier.default_options()
    opts.unescape_strings = True
    opts.eval_code = False
    return jsbeautifier.beautify(source, opts)

def find_stream_urls(js: str) -> list[str]:
    patterns = [
        r'"(https?://[^"]+\.m3u8[^"]*)"',
        r"'(https?://[^']+\.m3u8[^']*)'"
        r'src:\s*"(https://[^"]+\.mp4[^"]*)"',
        r'file:\s*"(https://[^"]+)"',
    ]
    results = []
    for p in patterns:
        results.extend(re.findall(p, js))
    return list(set(results))
```

§5.3 - Playwright Stealth Scraper (Production Template)

Python - Production Playwright

```
# Install: pip install playwright playwright-stealth
# playwright install chromium

from playwright.async_api import async_playwright
from playwright_stealth import stealth_async
import asyncio

STEALTH_UA = (
    'Mozilla/5.0 (Windows NT 10.0; Win64; x64) '
    'AppleWebKit/537.36 (KHTML, like Gecko) '
    'Chrome/125.0.0.0 Safari/537.36'
)

async def stealth_get_stream(url: str, referer: str) -> str:
    m3u8_urls = []
    async with async_playwright() as pw:
        browser = await pw.chromium.launch(
            headless=True,
            args=[
                '--no-sandbox',
                '--disable-setuid-sandbox',
                '--disable-blink-features=AutomationControlled',
                '--disable-dev-shm-usage',
            ]
        )
        ctx = await browser.new_context(
            user_agent=STEALTH_UA,
            viewport={'width': 1280, 'height': 720},
            extra_http_headers={
                'Accept-Language': 'en-US,en;q=0.9',
                'Referer': referer,
            }
        )
        page = await ctx.new_page()
        await stealth_async(page)  # inject stealth patches

        def on_request(req):
            if '.m3u8' in req.url:
                m3u8_urls.append(req.url)

        page.on('request', on_request)
        await page.goto(url, wait_until='domcontentloaded', timeout=25000)

        # Try to click play button
        try:
            await page.click('video, .play-btn, [class*=play], [id*=play]',
                             timeout=5000)
        except:
            pass
```

```
            await asyncio.sleep(6)
            await browser.close()

        return m3u8_urls[0] if m3u8_urls else None
```

§6 CDN & Stream Host Extraction

The video CDN hosts that actually serve the video bytes

All piracy sites ultimately funnel their content through a small set of video CDN hosts. These are the actual servers sending video bytes to users. Knowing which CDN a provider uses tells you exactly what extraction method to apply.

CDN Host	Domain Pattern	Extraction Method	Difficulty
StreamWish	streamwish.to / strwish.com	Regex: jwplayer sources[] in page JS	Easy
Filelions	filelions.live / lion.filelions.live	Regex: jwplayer sources[] in page JS	Easy
Streamtape	streamtape.com / streamtape.net	Regex: hidden input value + JS concat	Easy
DoodStream	doodstream.com / dood.pm	Intercept /pass_md5 XHR request	Medium
StreamSB	streamsb.com / sbplay.org	AES decryption of obfuscated URL	Hard
VidHide	vidhide.com / vidhide.art	Playwright intercept OR AES decrypt	Medium
VidPlay	vidplay.online / vidplay.site	Playwright intercept (uses captcha)	Hard
MixDrop	mixdrop.ag / mixdrop.to	JS eval unpack -> regex m3u8	Medium
GDFlix	new.gdflix.dad	POST to /api/direct with file_id	Medium
Cloudflare R2	r2.dev URLs	Direct - no extraction needed	Easy
BunnyNet CDN	*.b-cdn.net	Direct - often returned by jwplayer	Easy

StreamWish / Filelions Extractor (covers 60% of Indian sites)

Python - StreamWish Extractor

```
import re, requests

def extract_streamwish(embed_url: str, referer: str) -> str:
    headers = {
        'User-Agent': STEALTH_UA,
        'Referer': referer,
    }
    r = requests.get(embed_url, headers=headers, timeout=15)
    html = r.text

    # Method 1: jwplayer config in JS
    m = re.search(
        r'jwplayer\(.*?\)\.setup\(\{.*?sources:\s*\[\{.*?'
        r'file:\s*["\']([^"\'>]+\.m3u8[^"\']*)["\']',
        html, re.DOTALL
    )
    if m: return m.group(1)

    # Method 2: var sources = [{ file: '...' }]
    m = re.search(r'file:\s*["\']([^"\'>]+\.m3u8[^"\']*)["\']]', html)
    if m: return m.group(1)

    return None
```

Streamtape Extractor

Python - Streamtape Extractor

```
import re, requests

def extract_streamtape(embed_url: str) -> str:
    r = requests.get(embed_url, headers={'User-Agent': STEALTH_UA})
    html = r.text

    # Streamtape uses two JS vars that must be concatenated
    m1 = re.search(r"document\.getElementById\('norobotlink'\)\.innerHTML\s*=\s*([^;]+);", html)
    if not m1: return None

    # The value is: 'https://streamtape.com/get_video?id=...' + substr_of_another_var
    m2 = re.search(r"videolink\.substring\((\d+)\)", html)
    m3 = re.search(r'var\s+\w+\s*=\s*["\']([^"\']+)["\']\s*\+', html)

    if m3 and m2:
        base = m3.group(1)
        offset = int(m2.group(1))
        return 'https:' + base[offset:]
    return None
```

DoodStream Extractor

Python - DoodStream Extractor

```
import re, requests, time, random, string

def extract_doodstream(embed_url: str) -> str:
    # DoodStream serves stream via a /pass_md5/ XHR that returns a base URL
    # Then appends a token string + timestamp
    s = requests.Session()
    s.headers['User-Agent'] = STEALTH_UA

    r = s.get(embed_url.replace('/d/', '/e/'))
    token_match = re.search(r"'/pass_md5/([^']+)'", r.text)
    if not token_match: return None

    pass_path = '/pass_md5/' + token_match.group(1)
    base_resp = s.get('https://doodstream.com' + pass_path,
                      headers={'Referer': embed_url})
    base_url  = base_resp.text.strip()

    rand_str = ''.join(random.choices(string.ascii_lowercase + string.digits, k=10))
    timestamp = int(time.time() * 1000)
    return f'{base_url}{rand_str}?token=...&expiry={timestamp}'
```

§7 Custom HLS Video Player - Build Spec

React component using hls.js - full controls, quality switching, subtitles

Build the player as a self-contained React component at client/src/components/player/AllratedPlayer.tsx. It receives a streamUrl (m3u8 or mp4), an array of subtitle tracks, and metadata. It must handle: HLS adaptive streaming, quality level switching, playback speed, subtitle track selection, fullscreen, picture-in-picture, keyboard shortcuts, and resume from last position via the existing WatchProgress backend.

Package Installation

Shell - Install

```
# Inside client/ directory (NOT repo root)
cd client
npm install hls.js
npm install @types/hls.js --save-dev

# Optional but recommended - lightweight UI skin
npm install plyr
npm install @types/plyr --save-dev

# Note: hls.js ships its own types in hls.js v1.x+ - no @types needed for v1+
```

AllratedPlayer.tsx - Complete Component

TypeScript - Component Interface

```tsx
import React, { useRef, useEffect, useState, useCallback } from 'react';
import Hls, { Level } from 'hls.js';

interface SubtitleTrack {
  label: string;
  lang: string;
  url: string;  // .vtt URL
}

interface StreamInfo {
  url: string;          // .m3u8 or .mp4
  headers?: Record<string, string>;  // e.g. Referer
  provider: string;     // e.g. 'hdhub4u'
  quality?: string;     // e.g. '1080p'
  subtitles?: SubtitleTrack[];
}

interface AllratedPlayerProps {
  tmdbId: number;
  type: 'movie' | 'tv';
  season?: number;
  episode?: number;
  title: string;
  posterUrl?: string;
  initialProgress?: number;  // seconds  -  from WatchProgress
  onProgress?: (seconds: number, duration: number) => void;
}
```

TypeScript - State + Data Fetch

```
export const AllratedPlayer: React.FC<AllratedPlayerProps> = ({
  tmdbId, type, season, episode, title, posterUrl,
  initialProgress = 0, onProgress,
}) => {
  const videoRef   = useRef<HTMLVideoElement>(null);
  const hlsRef     = useRef<Hls | null>(null);
  const [stream, setStream]       = useState<StreamInfo | null>(null);
  const [loading, setLoading]     = useState(true);
  const [error, setError]         = useState<string | null>(null);
  const [levels, setLevels]       = useState<Level[]>([]);
  const [curLevel, setCurLevel]   = useState(-1);  // -1 = auto
  const [playing, setPlaying]     = useState(false);
  const [muted, setMuted]         = useState(false);
  const [volume, setVolume]       = useState(1);
  const [currentTime, setCurrentTime] = useState(0);
  const [duration, setDuration]   = useState(0);
  const [showControls, setShowControls] = useState(true);
  const [subtitleIdx, setSubtitleIdx]   = useState(-1);
  const [speed, setSpeed]         = useState(1);

  // -- Fetch stream URL from backend --
  useEffect(() => {
    const params = new URLSearchParams({
      type,
      ...(season != null  ? { s: String(season)  } : {}),
      ...(episode != null ? { e: String(episode) } : {}),
    });
    fetch(`/api/stream/${tmdbId}?${params}`)
      .then(r => r.ok ? r.json() : Promise.reject(r.statusText))
      .then(data => { setStream(data); setLoading(false); })
      .catch(err => { setError(String(err)); setLoading(false); });
  }, [tmdbId, type, season, episode]);
```

TypeScript - HLS Attach

```
// -- Attach hls.js once stream URL is known --
useEffect(() => {
  if (!stream || !videoRef.current) return;
  const video = videoRef.current;

  if (stream.url.includes('.m3u8') && Hls.isSupported()) {
    const hls = new Hls({
      xhrSetup: (xhr, url) => {
        if (stream.headers?.Referer)
          xhr.setRequestHeader('Referer', stream.headers.Referer);
      },
      enableWorker: true,
      lowLatencyMode: false,
      backBufferLength: 90,
    });
    hls.loadSource(stream.url);
    hls.attachMedia(video);
    hls.on(Hls.Events.MANIFEST_PARSED, (_, data) => {
      setLevels(data.levels);
      video.currentTime = initialProgress;
      video.play().catch(() => {});
      setPlaying(true);
    });
    hls.on(Hls.Events.LEVEL_SWITCHED, (_, { level }) => setCurLevel(level));
    hls.on(Hls.Events.ERROR, (_, data) => {
      if (data.fatal) setError('Stream failed  -  trying next provider...');
    });
    hlsRef.current = hls;
  } else {
    // Native HLS (Safari) or direct MP4
    video.src = stream.url;
    video.currentTime = initialProgress;
    video.play().catch(() => {});
    setPlaying(true);
  }

  return () => { hlsRef.current?.destroy(); };
}, [stream]);
```

TypeScript - Controls Logic

```
  // -- Subtitle injection --
  const setSubtitle = (idx: number) => {
    const video = videoRef.current;
    if (!video) return;
    // Hide all tracks
    Array.from(video.textTracks).forEach(t => (t.mode = 'hidden'));
    if (idx >= 0 && stream?.subtitles?.[idx]) {
      const track = video.textTracks[idx];
      if (track) track.mode = 'showing';
    }
    setSubtitleIdx(idx);
  };

  // -- Progress reporting --
  const handleTimeUpdate = () => {
    const v = videoRef.current;
    if (!v) return;
    setCurrentTime(v.currentTime);
    onProgress?.(v.currentTime, v.duration);
  };

  // -- Quality switching --
  const setQuality = (level: number) => {
    if (hlsRef.current) hlsRef.current.currentLevel = level;
    setCurLevel(level);
  };

  if (loading) return <div className='player-loading'>Finding stream...</div>;
  if (error)   return <div className='player-error'>{error}</div>;
```

TypeScript - JSX Return

```
  return (
    <div className='allrated-player'
         onMouseMove={() => setShowControls(true)}>
      {/* Subtitle tracks  -  add <track> elements for each */}
      <video
        ref={videoRef}
        className='video-element'
        onTimeUpdate={handleTimeUpdate}
        onDurationChange={e => setDuration(e.currentTarget.duration)}
        onPlay={() => setPlaying(true)}
        onPause={() => setPlaying(false)}
        crossOrigin='anonymous'
        playsInline
      >
        {stream?.subtitles?.map((sub, i) => (
          <track key={i} kind='subtitles' label={sub.label}
                 srcLang={sub.lang} src={sub.url}
                 default={i === 0} />
        ))}
      </video>

      {showControls && (
        <div className='player-controls'>
          <div className='progress-bar'>
            <input type='range' min={0} max={duration} value={currentTime}
                   onChange={e => {
                     const v = videoRef.current;
                     if (v) v.currentTime = Number(e.target.value);
                   }} />
          </div>
          <button onClick={() => playing ? videoRef.current?.pause()
                                         : videoRef.current?.play()}>
            {playing ? '?' : '>'}
          </button>
          {/* Quality selector */}
          <select value={curLevel} onChange={e => setQuality(Number(e.target.value))}>
            <option value={-1}>Auto</option>
            {levels.map((l, i) => (
              <option key={i} value={i}>{l.height}p</option>
            ))}
          </select>
          {/* Speed selector */}
          <select value={speed} onChange={e => {
            const s = Number(e.target.value);
            setSpeed(s);
            if (videoRef.current) videoRef.current.playbackRate = s;
          }}>
            {[0.5, 0.75, 1, 1.25, 1.5, 2].map(s => (
              <option key={s} value={s}>{s}x</option>
```

```
            ))}
          </select>
          <button onClick={() => videoRef.current?.requestPictureInPicture()}>
            PiP
          </button>
          <button onClick={() => videoRef.current?.requestFullscreen()}>
            ?
          </button>
        </div>
      )}
    </div>
  );
};
```

§8 Backend API - Server-Side Routes

Express routes + StreamResolver service on Railway

File Structure to Create

File Structure

```
server/src/
+--- routes/
|   +--- stream.ts          <- new  -  /api/stream/:tmdbId
+--- services/
|   +--- streamResolver.ts  <- orchestrates provider fallback chain
|   +--- providers/
|   |   +--- index.ts       <- exports all providers
|   |   +--- consumet.ts    <- anime via Consumet API
|   |   +--- hdhub4u.ts     <- Indian sites
|   |   +--- kissKh.ts      <- K-drama
|   |   +--- rive.ts        <- RiveStream fallback
|   |   +--- ytdlp.ts       <- yt-dlp universal fallback
|   +--- cdnExtractors/
|       +--- streamwish.ts
|       +--- streamtape.ts
|       +--- doodstream.ts
|       +--- index.ts
+--- utils/
    +--- streamCache.ts     <- node-cache wrapper (6h TTL)
```

stream.ts - Express Route

TypeScript - stream.ts route

```
import { Router, Request, Response } from 'express';
import { resolveStream } from '../services/streamResolver';

const router = Router();

/**
 * GET /api/stream/:tmdbId?type=movie|tv&s=1&e=1
 * Returns: { streamUrl, provider, quality, subtitles, headers }
 */
router.get('/:tmdbId', async (req: Request, res: Response) => {
  const tmdbId = parseInt(req.params.tmdbId, 10);
  const { type = 'movie', s, e } = req.query as Record<string, string>;

  if (!tmdbId || isNaN(tmdbId)) {
    return res.status(400).json({ error: 'Invalid tmdbId' });
  }

  try {
    const result = await resolveStream({
      tmdbId,
      type: type as 'movie' | 'tv',
      season: s ? parseInt(s, 10) : undefined,
      episode: e ? parseInt(e, 10) : undefined,
    });
    res.json(result);
  } catch (err: any) {
    console.error('[stream] resolve failed:', err.message);
    res.status(502).json({ error: 'No stream found', detail: err.message });
  }
});

export default router;
```

streamResolver.ts - Fallback Chain Orchestrator

TypeScript - streamResolver.ts

```
import NodeCache from 'node-cache';
import { consumetProvider }   from './providers/consumet';
import { hdHub4uProvider }    from './providers/hdhub4u';
import { kissKhProvider }     from './providers/kissKh';
import { riveProvider }       from './providers/rive';
import { ytdlpProvider }      from './providers/ytdlp';

const cache = new NodeCache({ stdTTL: 6 * 60 * 60 }); // 6 hours

interface ResolveInput {
  tmdbId: number; type: 'movie' | 'tv';
  season?: number; episode?: number;
}

// Provider priority order  -  tune this based on reliability
const PROVIDERS = [
  consumetProvider,   // #1  -  anime API, fastest
  hdHub4uProvider,    // #2  -  movies + shows
  kissKhProvider,     // #3  -  drama
  riveProvider,       // #4  -  global fallback
  ytdlpProvider,      // #5  -  nuclear fallback
];

export async function resolveStream(input: ResolveInput) {
  const cacheKey = `stream:${input.tmdbId}:${input.type}:${input.season}:${input.episode}`;
  const cached = cache.get(cacheKey);
  if (cached) return cached;

  for (const provider of PROVIDERS) {
    try {
      const result = await provider.resolve(input);
      if (result?.streamUrl) {
        cache.set(cacheKey, result);
        return result;
      }
    } catch (err: any) {
      console.warn(`[${provider.name}] failed:`, err.message);
    }
  }
  throw new Error('All providers exhausted');
}
```

consumet.ts - Provider (Anime)

TypeScript - consumet.ts

```
import axios from 'axios';

const BASE = process.env.CONSUMET_API_URL || 'https://consumet.zendax.tech';

export const consumetProvider = {
  name: 'consumet',
  async resolve({ tmdbId, type, season, episode }) {
    // Only handle anime TV shows
    if (type === 'movie') return null;

    // Map TMDB -> Consumet episode ID
    const info = await axios.get(
      `${BASE}/meta/tmdb/info/${tmdbId}?type=tv`
    );
    const eps = info.data?.episodes || [];
    const ep  = eps.find((e: any) =>
      e.season === season && e.number === episode
    );
    if (!ep) return null;

    // Get stream URLs
    const stream = await axios.get(
      `${BASE}/meta/tmdb/watch/${ep.id}?id=${tmdbId}`
    );
    const src = stream.data?.sources?.find((s: any) => s.isM3U8);
    if (!src) return null;

    return {
      streamUrl: src.url,
      provider: 'consumet',
      quality: src.quality || 'auto',
      headers: stream.data?.headers || {},
      subtitles: (stream.data?.subtitles || []).map((s: any) => ({
        label: s.lang, lang: s.lang, url: s.url,
      })),
    };
  },
};
```

Register Route in server/src/index.ts

TypeScript - Route Registration

```
// In server/src/index.ts or server/src/app.ts:
import streamRouter from './routes/stream';

// Add alongside existing routes:
app.use('/api/stream', streamRouter);
```

§9 Frontend Integration - Allrated Client

Where to add the player and how to wire it up in TitleDetail

The player currently lives in client/src/pages/TitleDetail.tsx. It renders a ScreenScape iframe inside a conditional block that checks if a screenscape embed URL is available. Replace that block with the AllratedPlayer component. The WatchProgress integration already exists - hook onProgress into the existing saveProgress() call.

TitleDetail.tsx - Replace ScreenScape block

TypeScript - TitleDetail change

```
// -- BEFORE (remove this) ------------------------------------------------
import { ScreenScapePlayer } from '../components/player/ScreenScapePlayer';

{embedUrl && (
  <ScreenScapePlayer
    embedUrl={embedUrl}
    title={title.title}
  />
)}

// -- AFTER (replace with this) -------------------------------------------
import { AllratedPlayer } from '../components/player/AllratedPlayer';

{title && (
  <AllratedPlayer
    tmdbId={title.tmdbId}
    type={title.type as 'movie' | 'tv'}
    season={currentSeason}         // existing state
    episode={currentEpisode}       // existing state
    title={title.title}
    posterUrl={title.posterPath
      ? `https://image.tmdb.org/t/p/w780${title.posterPath}`
      : undefined}
    initialProgress={watchProgress?.position ?? 0}
    onProgress={(seconds, duration) => {
      saveProgress(title.id, seconds, duration); // existing function
    }}
  />
)}
```

CSS - Player Styles (add to client/src/styles/player.css)

CSS - Player Styles

```
.allrated-player {
  position: relative;
  width: 100%;
  aspect-ratio: 16 / 9;
  background: #000;
  border-radius: 8px;
  overflow: hidden;
  font-family: var(--font-sans);
}

.allrated-player .video-element {
  width: 100%;
  height: 100%;
  object-fit: contain;
}

.allrated-player .player-controls {
  position: absolute;
  bottom: 0; left: 0; right: 0;
  padding: 12px 16px;
  background: linear-gradient(transparent, rgba(0,0,0,0.85));
  display: flex;
  align-items: center;
  gap: 12px;
  transition: opacity 0.3s;
}

.allrated-player .progress-bar input[type=range] {
  flex: 1;
  height: 4px;
  accent-color: #E03C28;
  cursor: pointer;
}

.allrated-player .player-loading,
.allrated-player .player-error {
  display: flex;
  align-items: center;
  justify-content: center;
  height: 100%;
  color: #fff;
  font-size: 1rem;
  background: #0a0a14;
}

.allrated-player .player-error { color: #ff6b6b; }
```

§10 Fallback Chain & Error Handling

How to handle provider failures gracefully

The fallback chain is the most important reliability feature. No single provider has 100% uptime or 100% coverage. The system must silently try the next provider on any failure - network error, missing title, blocked by geo-restriction, or bad stream quality.

Fallback Priority Matrix

Priority	Provider	Best For	Fallback Trigger
1	Consumet API	Anime	Returns null or throws
2	HDHub4U scraper	Movies + TV (English + Hindi)	Search returns no results
3	VegaMovies scraper	Movies (4K available)	HDHub4U failed
4	KissKH API	K-drama, C-drama	Type is tv + Korean/Asian
5	RiveStream embed	Any content (generic)	All above failed
6	yt-dlp universal	Anything yt-dlp knows	RiveStream failed
7	Error response	-	All 6 failed - return 502

Stream Validity Check

Before returning a stream URL to the client, always validate it is actually accessible. A scraper can return a URL that has already expired or is geo-blocked.

TypeScript - Stream Validation

```
import axios from 'axios';

async function validateStreamUrl(url: string, headers?: Record<string, string>): Promise<boolean> {
  try {
    const resp = await axios.head(url, {
      headers: headers ?? {},
      timeout: 8000,
      maxRedirects: 5,
    });
    const ct = resp.headers['content-type'] || '';
    // Valid if content-type is video/* or application/x-mpegURL
    return ct.includes('video') || ct.includes('mpegURL') || ct.includes('octet-stream');
  } catch {
    return false;
  }
}

// In streamResolver.ts  -  add validation before caching:
const isValid = await validateStreamUrl(result.streamUrl, result.headers);
if (!isValid) continue; // skip this provider
cache.set(cacheKey, result);
return result;
```

Client-Side Retry on Stream Error

TypeScript - Client Retry

```
// In AllratedPlayer.tsx  -  handle HLS fatal errors by retrying backend
hls.on(Hls.Events.ERROR, async (_, data) => {
  if (data.fatal) {
    // Mark current provider as failed, request a different one
    const params = new URLSearchParams({
      type, skip: stream!.provider,  // tell backend to skip this provider
      ...(season != null ? { s: String(season) } : {}),
      ...(episode != null ? { e: String(episode) } : {}),
    });
    try {
      const r = await fetch(`/api/stream/${tmdbId}?${params}`);
      const newStream = await r.json();
      setStream(newStream);  // triggers useEffect to re-attach hls
    } catch {
      setError('All streams failed. Please try again later.');
    }
  }
});
```

§11 Anti-Bot Evasion Techniques

Bypassing Cloudflare, PerimeterX, and bot detection

- Use playwright-stealth (pip install playwright-stealth). It patches >30 browser fingerprint properties that headless Chrome normally exposes - navigator.webdriver, chrome.runtime, window.cdc_*, etc.
- Rotate User-Agent strings. Maintain a pool of 10+ real Chrome UA strings from Windows, Mac, and Android. Never use the default Playwright UA.
- Set realistic browser headers: Accept-Language, Accept-Encoding, Sec-Ch-Ua, Sec-Ch-Ua-Platform, Sec-Fetch-Site, Sec-Fetch-Mode.
- Add random delays between requests. Use random.uniform(1.5, 4.0) seconds between page loads. Never make concurrent requests to the same domain.
- Use residential proxy rotation for Cloudflare-protected sites. Services: BrightData, Oxylabs, Smartproxy. Add proxy config to Playwright context.
- For Cloudflare 5s challenge: use cloudscraper (pip install cloudscraper) for simple HTTP requests. For JS-challenge: must use Playwright with stealth.
- Cache aggressively. A 6-hour cache means you only scrape each title once per 6 hours. Most bot detection triggers on high request frequency, not single requests.

Shell + Python - Anti-Bot Libraries

```
pip install cloudscraper playwright-stealth fake-useragent

# cloudscraper for simple Cloudflare bypass (no JS challenge):
import cloudscraper
scraper = cloudscraper.create_scraper(
    browser={'browser': 'chrome', 'platform': 'windows', 'mobile': False}
)
response = scraper.get('https://hdhub4u.cl/?s=Inception+2010')

# fake-useragent for rotating UAs:
from fake_useragent import UserAgent
ua = UserAgent()
headers = {'User-Agent': ua.chrome}
```

§12 Subtitle Extraction & Loading

Getting English subtitles for every title

Subtitles come from two sources: the streaming site itself (embedded in the m3u8 or returned by the provider API), or from OpenSubtitles / Subscene as a fallback.

Source 1 - Provider Subtitles (Consumet, KissKH)

HTTP - Provider Subtitles

```
// Consumet returns subtitles directly:
{ subtitles: [{ url: 'https://...vtt', lang: 'English' }] }

// KissKH returns subtitles via API:
GET https://kisskh.ws/api/DramaList/Episode/{id}.png
-> { Video: '...m3u8', SubTitle: [{ land: 'en', src: '...vtt' }] }
```

Source 2 - OpenSubtitles REST API

HTTP - OpenSubtitles

```
# Free tier: 5 requests/second, 200/day  -  enough for a small app
GET https://api.opensubtitles.com/api/v1/subtitles
  ?tmdb_id={tmdbId}
  &type=movie
  &languages=en
  &order_by=download_count
Headers: { Api-Key: {OPENSUBTITLES_API_KEY} }

-> { data: [{ attributes: { files: [{ file_id, file_name }] } }] }

# Download the subtitle file
POST https://api.opensubtitles.com/api/v1/download
  { file_id: 12345 }
-> { link: 'https://...srt' }

# Convert SRT -> VTT for use in <track>:
# Use npm package: srt-to-vtt  OR  server-side Python: srt library
```

WARNING

Store OPENSUBTITLES_API_KEY as a Railway environment variable.

Never hardcode it. Add it via Railway dashboard -> Variables.

Sign up free at: opensubtitles.com/en/consumers

§13 Complete Implementation Checklist

Step-by-step for the agent - in order, no steps skippable

PHASE 1 - SETUP

1. Read this entire brief before writing a single line of code.
2. Inspect client/src/components/player/ for any existing player files.
3. Inspect server/src/routes/ for existing route registration pattern.
4. Inspect server/src/index.ts or app.ts to see how routes are added.
5. Install hls.js inside client/: cd client && npm install hls.js
6. Install node-cache inside server/: cd server && npm install node-cache
7. Add OPENSUBTITLES_API_KEY to Railway environment variables.
8. Add CONSUMET_API_URL to Railway env vars (or use default).

PHASE 2 - BACKEND

1. Create server/src/services/streamResolver.ts with fallback chain.
2. Create server/src/services/providers/consumet.ts - Consumet anime provider.
3. Create server/src/services/providers/hdhub4u.ts - Indian site scraper.
4. Create server/src/services/providers/kissKh.ts - K-drama API provider.
5. Create server/src/services/providers/rive.ts - RiveStream fallback.
6. Create server/src/services/providers/ytdlp.ts - yt-dlp nuclear fallback.
7. Create server/src/services/cdnExtractors/streamwish.ts
8. Create server/src/services/cdnExtractors/streamtape.ts
9. Create server/src/services/cdnExtractors/doodstream.ts
10. Create server/src/routes/stream.ts with GET /:tmdbId
11. Register /api/stream in server/src/index.ts or app.ts
12. Test: curl https://netflixallrated.up.railway.app/api/stream/27205?type=movie
13. Verify: response includes { streamUrl, provider, quality, subtitles }
14. Verify: second call returns cached result in < 50ms

PHASE 3 - PLAYER COMPONENT

1. Create client/src/components/player/AllratedPlayer.tsx - full component.
2. Create client/src/styles/player.css - all player styles.
3. Import player.css in AllratedPlayer.tsx.
4. Test player locally: render <AllratedPlayer tmdbId={550} type='movie' title='Fight Club' />
5. Verify: loading state shows while fetching stream.
6. Verify: hls.js attaches and plays the m3u8.
7. Verify: quality selector populates from HLS manifest levels.
8. Verify: subtitle tracks appear if provider returns them.
9. Verify: progress is reported via onProgress callback.

PHASE 4 - TITLE DETAIL INTEGRATION

1. Open client/src/pages/TitleDetail.tsx.
2. Find the existing ScreenScape iframe / ScreenScapePlayer block.
3. Replace with <AllratedPlayer> using the mapping in §9.
4. Wire initialProgress from existing watchProgress state.
5. Wire onProgress to existing saveProgress() function.
6. For TV shows: pass currentSeason and currentEpisode as season/episode props.

PHASE 5 - TESTING & VERIFICATION

1. Test a movie: navigate to any movie title detail page - player should load.
2. Test a TV show episode: navigate to a TV show, select an episode - player loads.
3. Test an anime title - Consumet provider should handle it.
4. Test a K-drama title - KissKH provider should handle it.
5. Simulate provider failure: temporarily break hdHub4u scraper URL. Verify next provider in chain is tried automatically.
6. Test subtitle display: verify English subtitle track loads and displays.
7. Test resume: play 30s of a movie, reload the page, verify it resumes at 30s.
8. Test quality switching: change quality level - video should switch without stopping.
9. Test PiP: click PiP button - player should enter picture-in-picture mode.
10. Test fullscreen: click fullscreen button - player should go fullscreen.
11. Run TypeScript check: cd server && npx tsc --noEmit (must have 0 errors).
12. Run TypeScript check: cd client && npx tsc --noEmit (must have 0 errors).
13. Commit to GitHub: git push origin main - Railway auto-deploys.
14. Verify deployed: curl the production /api/stream endpoint on Railway.

PHASE 6 - MONITORING (Optional but Recommended)

1. Add Winston/pino logging to streamResolver: log provider tried, time taken, result.
2. Add a /api/stream/stats route that returns cache hit rate and provider success rates.
3. Set up a Railway cron job to check provider health every 30 minutes.
4. Add Sentry error tracking to catch scraper failures in production.

You now have everything needed.

Provider list. Scraping methodology. CDN extractor code.
Reverse engineering toolkit. Player component. Backend routes.
Integration steps. Fallback chain. Anti-bot evasion. Subtitles.

Follow the phases in order. Test each phase before moving on.
The TypeScript checks must pass at zero errors before shipping.